

Rapport Final Global

Applications Distribuées adaptées au parcours Big Data

Étudiant	Hamza Mannad
Filière	3ème année Big Data
Module	Applications Distribuées — Parcours Big Data
Année universitaire	2025 – 2026

Table des matières

1. Introduction

2. Synthèse des Travaux Pratiques

TP1 — Pipeline ETL Météo & Irrigation

TP2 — Service d'ingestion IoT (POO, Validation, Sérialisation)

TP3 — Mini service d'ingestion IoT sur sockets TCP

TP4 — Mini RPC Data Service (Ingestion + Stats)

TP5 — Pipeline asynchrone d'ingestion IoT

TP6 — Mini Stream Processor IoT

TP7 — Mini broker partitionné avec consumers et offsets

TP8 — Architecture batch + stream, data lake et gouvernance

3. Analyse personnelle

4. Conclusion générale

1. Introduction

Ce rapport constitue la synthèse globale du travail réalisé tout au long du semestre dans le cadre du module **Applications Distribuées adaptées au parcours Big Data**, suivi en troisième année de la filière Big Data. L'objectif principal de ce module était de construire progressivement une compréhension profonde et pratique des architectures distribuées modernes, depuis les fondamentaux du traitement de données jusqu'aux paradigmes avancés du streaming et du partitionnement.

Le module s'est articulé autour de huit séances, chacune introduisant de nouveaux concepts et les mettant en pratique à travers un TP guidé. La progression pédagogique suivait une logique cumulative : chaque séance s'appuyait sur les acquis des précédentes pour introduire de nouvelles notions. Ainsi, nous avons débuté par un pipeline ETL classique avant d'aborder la programmation orientée objet appliquée aux contrats de messages, les communications réseau via sockets TCP et RPC HTTP, les architectures événementielles asynchrones, le stream processing avec gestion du temps, et enfin le partitionnement distribué.

L'ensemble des travaux pratiques a été réalisé exclusivement en Python standard (bibliothèque standard Python 3.11+), sans dépendances externes lourdes, afin de se concentrer sur la compréhension des mécanismes sous-jacents plutôt que sur la maîtrise d'outils spécifiques. Cette approche pédagogique s'est révélée particulièrement efficace pour construire des intuitions solides sur le fonctionnement réel des plateformes Big Data comme Kafka, Spark ou Flink.

2. Synthèse des Travaux Pratiques

TP1 — Pipeline ETL Météo & Irrigation

Contexte	Traitement d'un jeu de données brutes de stations météo agricoles présentant de nombreuses erreurs.
Objectifs	Construire un pipeline ETL complet : Extract → Transform → Quality Report → Feature Engineering.
Outils	Python, pandas, numpy. Bibliothèque standard uniquement.
Démarche	Lecture du CSV brut, nettoyage en 9 étapes (coercion, déduplication, domaines, IQR, etc.).
Résultats	16 lignes en entrée → 13 lignes propres. 5 features créées. Toutes les assertions passées.

Ce premier TP a posé les bases du traitement de données structurées. La principale difficulté résidait dans la gestion correcte de la déduplication : la méthode `drop_duplicates()` doit exclure la colonne `id` pour détecter les vrais doublons métier, ce qui n'est pas intuitif de prime abord. La décision de conserver la valeur aberrante `wind_kmh=150` (vent de tempête physiquement plausible) illustre bien l'importance du jugement métier dans le nettoyage de données.

TP2 — Service d'ingestion IoT (POO, Validation, Sérialisation)

Contexte	Construction de la couche logicielle d'un service d'ingestion IoT avec des contrats de données stricts.
Objectifs	Maîtriser les dataclasses Python, la sérialisation JSON aller-retour, et les patterns de validation.
Outils	Python standard : dataclasses, json, uuid, logging.
Démarche	Définition de 4 dataclasses (<code>ValidationError</code> , <code>SensorReading</code> , <code>IngestRequest</code> , <code>IngestResponse</code>).
Résultats	5 readings → 1 accepté, 4 rejetés. 10 assertions passées. Logging sécurisé avec masquage.

Ce TP a introduit les fondements de la conception orientée objet appliquée aux pipelines de données. Le principe de validation cumulative (collecter toutes les erreurs plutôt que s'arrêter à la première) est un pattern essentiel dans les services Big Data réels, car il permet au producteur de corriger l'ensemble de ses erreurs en un seul aller-retour. La sécurité des logs (masquage de `api_key`, sanitization anti-injection) a également été une découverte importante.

TP3 — Mini service d'ingestion IoT sur sockets TCP

Contexte	Communication réseau réelle entre un client et un serveur IoT via le protocole TCP, avec des données binaires.
Objectifs	Comprendre les sockets TCP, gérer le partial read, implémenter un protocole applicatif simple.

Outils	Python standard : socket, json, threading, uuid.
Démarche	Implémentation du protocole NDJSON (encode_message, decode_message, recv_line)
Résultats	10 lectures → 5 acceptées, 5 rejetées. Pipeline complet encode → validate → decode

La gestion du partial read TCP a été la difficulté principale de ce TP. Dans un flux TCP, les données n'arrivent pas forcément en une seule fois : la fonction `recv_line()` accumule les octets dans un buffer jusqu'à trouver un saut de ligne. Ce pattern est fondamental et se retrouve dans tous les protocoles textuels (HTTP/1.1, SMTP, Redis, etc.). Le choix du format NDJSON (une ligne = un objet JSON complet) simplifie grandement le framing et est devenu un standard de facto dans les pipelines de streaming.

TP4 — Mini RPC Data Service (Ingestion + Stats)

Contexte	Création d'un service RPC complet sur HTTP : routeur de méthodes, threading, métrique
Objectifs	Comprendre le pattern RPC, implémenter un routeur extensible, gérer la concurrence
Outils	Python standard : http.server, ThreadingHTTPServer, json, threading, statistics.
Démarche	4 méthodes RPC (health.ping, ingest.batch, stats.daily_summary, stats.top_sensors)
Résultats	10 lectures → 6 acceptées, 4 rejetées. Métriques exportées (run_report.json). 10 ass

Ce TP a mis en évidence l'importance du pattern Open/Closed dans la conception d'un routeur : pour ajouter une nouvelle méthode, on enregistre simplement une fonction dans le registry sans modifier le handler HTTP. Le `ThreadingHTTPServer` permet de gérer plusieurs requêtes simultanées, illustrant concrètement la concurrence dans un service web. La stratégie de retry avec backoff exponentiel côté client est un mécanisme essentiel de résilience dans les architectures distribuées.

TP5 — Pipeline asynchrone d'ingestion IoT

Contexte	Architecture producteur/consommateur avec file bornée (backpressure), retry, dead-l
Objectifs	Maîtriser queue.Queue, threading, la gestion de la backpressure et les patterns de ré
Outils	Python standard : queue, threading, csv, statistics, logging.
Démarche	3 producteurs en rafales (450 messages, 10% invalides), 2 workers initiaux + scale-o
Résultats	~413 acceptés / ~37 DLQ / ~74 retries. Débit ~69 msg/s. Agrégation par capteur exp

Ce TP a représenté un saut conceptuel important : le passage d'un traitement séquentiel à un traitement concurrent avec backpressure. La queue bornée (maxsize=50) joue un rôle

de régulateur : si les workers n'arrivent pas à suivre, les producteurs sont naturellement ralentis voire bloqués. La dead-letter queue garantit qu'aucun message n'est silencieusement perdu : les échecs définitifs sont tracés et peuvent être analysés a posteriori.

TP6 — Mini Stream Processor IoT

Contexte	Pipeline streaming avec fenêtres tumbling, keyed state, watermark, gestion de la latence
Objectifs	Comprendre la distinction event time / processing time, implémenter un processeur stream
Outils	Python standard : queue, threading, json, pathlib, datetime.
Démarche	EventSource (thread daemon + sentinel), StreamProcessor (keyed state par (sensor_id, timestamp))
Résultats	13 événements → 10 fenêtres flushées, 2 late-accepted, 1 dropped (evt-011, lateness 120s)

La notion de watermark est l'une des plus complexes de tout le module. Elle représente une estimation du temps jusqu'auquel tous les événements ont été reçus, permettant de décider quand fermer une fenêtre temporelle. La politique de lateness (tolérer jusqu'à 120 secondes de retard avant de rejeter un événement) illustre le compromis fondamental du stream processing : exactitude versus latence. Plus on tolère de retard, plus les résultats sont complets, mais plus ils arrivent tard.

TP7 — Mini broker partitionné avec consumers et offsets

Contexte	Simulation complète d'un pipeline Big Data distribué : broker partitionné, consumer group, offsets
Objectifs	Comprendre le partitionnement par clé de hash, l'assignation round-robin des partitions aux consumers
Outils	Python standard : hashlib, threading, queue, json, pathlib, collections.
Démarche	Broker en mémoire (N listes = N partitions), fonction partition_of() via SHA-256, OffsetManager pour gérer les offsets
Résultats	Distribution équilibrée vérifiée, hotspot sensor-HOT-01 observé, offsets.json valide, rejeu réussi

Ce TP est celui qui m'a le plus éclairé sur le fonctionnement interne de Kafka. Réaliser qu'un broker n'est fondamentalement que N listes indépendantes avec une fonction de routage par hash a été un déclic important. L'écriture atomique des offsets via un fichier temporaire suivi d'un rename() est un pattern de fiabilité essentiel : en cas de crash, le fichier final est toujours cohérent. La distinction at-least-once (commit après traitement) vs at-most-once (commit avant) est désormais parfaitement claire.

TP8 — Architecture batch + stream, data lake et gouvernance

Contexte	Intégration des architectures lambda et kappa, organisation d'un data lake en zones (
Objectifs	Comprendre la cohabitation batch/stream, la structuration d'un data lake, les schéma
Outils	Python standard, concepts d'architecture (lambda, kappa), gouvernance des données
Démarche	Simulation d'un pipeline dual (batch historique + stream temps réel), organisation en
Résultats	Architecture complète batch+stream opérationnelle, data lake organisé en zones, gou

Ce dernier TP a apporté une vision architecturale globale qui donne du sens à l'ensemble du semestre. L'architecture lambda (deux chaînes parallèles : batch pour l'exactitude, stream pour la rapidité) et l'architecture kappa (tout traiter comme un stream rejouable) répondent à des besoins réels différents. La structuration du data lake en zones (raw → curated → refined) reflète la réalité des grandes organisations de données.

3. Analyse personnelle

Difficultés rencontrées

La principale difficulté technique rencontrée tout au long du semestre a été la gestion des imports Python dans les projets multi-fichiers. Dès le TP3, l'utilisation des imports relatifs (`from .module import ...`) nécessite de lancer les scripts via `python -m src.main` plutôt que `python src/main.py` directement. Cette subtilité, qui paraît anodine, a causé des erreurs `ModuleNotFoundError` répétées et a exigé de bien comprendre le système de packages Python.

La deuxième difficulté majeure a été la compréhension des mécanismes de concurrence. Dans les TPs 5, 6 et 7, la coordination entre threads via des `Event`, `Lock` et `Queue` nécessite une rigueur importante : un arrêt propre (`stop_event + join()`) mal implémenté peut causer des `deadlocks` ou des pertes silencieuses de données. La notion de `thread-safety`, en particulier pour les structures de données partagées, a demandé du temps pour être bien assimilée.

La compréhension de l'`event time` versus le `processing time` (TP6) a également été conceptuellement difficile. Admettre qu'un événement peut arriver avec plusieurs minutes de retard et décider s'il faut l'accepter ou le rejeter selon un `watermark` est un raisonnement qui s'éloigne de la logique séquentielle habituelle.

Solutions adoptées

Pour les problèmes d'imports, la solution adoptée a été d'ajouter en tête de chaque `main.py` une manipulation explicite de `sys.path` permettant le lancement direct via `python src/main.py`. Cette approche, bien que non idiomatique, simplifie grandement l'utilisation pour un utilisateur non familier avec les packages Python.

Pour la concurrence, l'adoption systématique du pattern `stop_event` (`threading.Event`) partagé entre tous les threads, combiné à un `join()` avec `timeout`, a permis des arrêts propres dans tous les TPs. L'utilisation de structures `thread-safe` (`queue.Queue`, `threading.Lock` autour des dict et listes) a résolu les conditions de course.

Pour le `stream processing`, la construction de tests unitaires sur des événements synthétiques (incluant des événements volontairement en retard et des cas limites) a permis de valider le comportement du `watermark` et de la politique de `lateness` de façon rigoureuse.

Apprentissages acquis

- La conception de contrats de messages typés (`dataclasses`) est la fondation de tout pipeline de données robuste : elle force à clarifier ce qu'on échange avant même

d'écrire la logique.

- La validation cumulative (collecter toutes les erreurs) est supérieure à la validation court-circuit dans un contexte de service distribué.
- Un broker partitionné n'est fondamentalement qu'un ensemble de files indexées avec une fonction de routage déterministe — comprendre cela démystifie des outils comme Kafka.
- L'écriture atomique (fichier .tmp + rename) est le pattern de fiabilité minimal pour tout état persistant dans un pipeline distribué.
- Le choix de la clé de partition est la décision architecturale la plus importante d'un pipeline Big Data : une mauvaise clé crée des hotspots qui annulent tous les bénéfices du parallélisme.
- La distinction at-least-once / at-most-once dépend uniquement du moment où l'on committe l'offset par rapport au traitement : avant (at-most-once, risque de perte) ou après (at-least-once, risque de duplication).
- Le watermark est une heuristique, pas une certitude : accepter la lateness jusqu'à un certain seuil est un compromis entre complétude et latence des résultats.
- Les architectures lambda et kappa répondent à des besoins réels différents et chaque organisation doit choisir selon ses contraintes de fraîcheur, d'exactitude et de complexité opérationnelle.

4. Conclusion générale

Ce semestre de module Applications Distribuées a constitué une progression pédagogique remarquablement cohérente, du traitement de données brutes à la conception d'architectures distribuées complexes. En huit séances, nous avons parcouru l'essentiel des briques constitutives d'un système Big Data moderne : ETL, contrats de messages, communication réseau, RPC, traitement asynchrone, stream processing avec gestion du temps, partitionnement distribué et gouvernance.

L'approche pédagogique choisie — tout implémenter en Python standard sans frameworks externes — s'est révélée particulièrement efficace. En construisant soi-même un broker partitionné, un processeur de fenêtres temporelles ou un gestionnaire d'offsets, on acquiert une compréhension profonde des mécanismes qui, dans des outils comme Kafka, Spark Streaming ou Flink, restent généralement cachés derrière des APIs de haut niveau. Cette compréhension est précieuse pour diagnostiquer des problèmes de performance, comprendre les garanties offertes et prendre des décisions architecturales éclairées.

Le module a également développé des compétences transversales importantes : la discipline du code modulaire et bien documenté, la rigueur de la gestion des erreurs et des cas limites, l'importance de l'observabilité (logs structurés, métriques, dashboards) et la culture du test systématique via des assertions. Ces compétences sont directement valorisables dans un contexte professionnel de data engineering.

En perspective, ce module constitue une excellente préparation à l'utilisation des plateformes Big Data professionnelles (Apache Kafka, Apache Spark, Apache Flink, Databricks). Les concepts maîtrisés ici — partitions, offsets, consumer groups, fenêtres temporelles, watermarks, checkpoints — sont exactement les mêmes que ceux manipulés au quotidien par les ingénieurs data. La transition vers ces outils sera désormais une affaire de syntaxe et d'APIs, non plus de concepts fondamentaux.

En conclusion, ce module a pleinement rempli son objectif : transformer une intuition vague sur le Big Data en une compréhension concrète, opérationnelle et démystifiée des systèmes distribués modernes.